第十四届“恩智浦”杯全国大学生
智能汽车竞赛

技 术 报 告



学　　校：　青岛科技大学

队伍名称：　Dragon

参赛队员：　刘志昂　李圣坤　夏建红

带队教师：　陈为

　　刘海波

关于技术报告和研究论文使用授权的说明

本人完全了解第 14 届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦半导体公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：刘志强

带队教师签名：刘海波

日　　期：2019.8.19

目 录

第一章 引言.....1
第二章 系统设计.....2
2.1 系统总体方案.....2
2.2 系统设计方案.....2
第三章 智能车机械结构调整与优化.....4
3.1 电磁传感器.....4
3.1.1 电磁传感器选择.....4
3.1.2 电磁传感器布局.....4
3.3 速度传感器.....5
3.4 车模机械调整.....5
3.4.1 车模前轮转向优化.....5
3.4.2 后轮转向改装.....6
第四章 硬件电路设计说明.....7
4.1 单片机最小系统.....7
4.2 电源模块.....7
4.3 运放模块.....8
4.4 电机驱动模块.....8
第五章 软件设计.....10
5.1 软件开发平台.....10
5.2 软件系统总体设计.....10
5.3 磁场信息获取.....13
5.4 控制算法.....13
5.4.1 转向控制——位置式 PID.....13
5.4.2 速度控制——增量式 PID 算法.....15
第六章 车模技术参数.....16
第七章 总结.....17
7.1 存在的问题.....17
7.1.1 机械方面.....17
7.1.2 硬件方面.....17
7.2 心得体会.....17
参考文献.....18
附录 源代码.....19

第一章 引言

全国大学生“恩智浦”杯智能汽车竞赛是以“立足培养，重在参与，鼓励探索，追求卓越”为指导思想，旨在促进高等学校素质教育，培养大学生的综合知识运用能力、基本工程实践能力和创新意识，激发大学生从事科学研究与探索的兴趣和潜能，倡导理论联系实际、求真务实的学风和团队协作的人文精神，为优秀人才的脱颖而出创造条件。

该竞赛由竞赛秘书处设计、规范标准硬软件技术平台，竞赛过程包括理论设计、实际制作、整车调试、现场比赛等环节，要求学生组成团队，协同工作，初步体会一个工程性的研究开发项目从设计到实现的全过程。该竞赛融科学性、趣味性和观赏性为一体，以迅猛发展、前景广阔的汽车电子为背景，涵盖自动控制、模式识别、传感技术、电子、电气、计算机、机械与汽车等多学科专业的创意性比赛。该竞赛规则透明，评价标准客观，坚持公开、公平、公正的原则，力求向健康、普康、持续的方向发展。

竞赛要求在规定的汽车模型平台上，使用恩智浦半导体公司的微控制器作为核心控制器，通过增加各种传感器、电机驱动模块以及编写相应控制程序，制作完成一个能够自主识别道路的智能汽车。本届竞赛分为小白四轮组、变形金刚三轮组、断桥相会双车组、飞毛腿节能组、横冲直闯信标组、室外越野电磁组、室外光电组七个组别和一个创意组别。

室外越野电磁组为今年的新兴组别，第十四届智能车竞赛室外电磁组比赛，比赛没有赛道，只有电磁引导线。赛道元素包括有直道、弯道、坡道、十字路口以及横断路障等。地内可能会存在高度不大于 2 厘米的硬质路坎、沙坑、深度不超过 2 厘米的水坑等。室外越野电磁组的车模使用科宇通博公司生产的 L 车模，采用恩智浦 32 位微控制器 MK60DN512ZVLQ10 作为核心控制单元，自主构思控制方案及系统设计，包括传感器信号采集处理、控制算法及执行、直流电机驱动及控制等，最终实现一套能够准确识别赛道，快速完成比赛的智能车系统。

在这份报告中，我们主要通过对机械上车辆结构和传感器布局、软件上控制算法和信号处理等方面的介绍，阐明我们的制作车辆在电感的排布方案、信号处理以及有效利用、避障方案、模糊动态加权巡线策略、参数调节的思路和过程、车上的电路硬件电路、车模改装等方面的创新和努力，集中了全队成员的心血和智慧，乐意公开与大家分享交流。

第二章 系统设计

本章主要介绍智能汽车系统总体设计方案和总体设计思路，在后面的章节中将整个系统分为机械结构、硬件模块、控制算法等三部分对智能汽车控制系统进行深入的介绍和分析。

2.1 系统总体方案

为控制车辆完成巡线、避障、停车任务，我们采用了电感、超声波模块、编码器等传感器、将采集到的信息经过硬件处理放大后交由 MCU 计算生产 PWM 控制舵机和驱动直流电机、并结合 OLED 显示屏、按键和蜂鸣器实现人机交互辅助控制和调试车辆。

2.2 系统设计方案

遵照竞赛规则规定，智能汽车系统采用恩智浦的 32 位微控制 MK60DN512ZVLQ10 单片机作为核心控制单元用于智能汽车系统的控制。虽然比赛要求放宽了对传感器的种类限制，但由于室外越野车要求在室外相对复杂的赛道环境中稳定运行，摄像头的处理难度大且不可靠，激光雷达无法采集到赛道上引导线的信息，故我们最终还是选择了传统的工字电感、赛车的位置信号由车体前方的工字电感和电容谐振后经过运放放大滤波，交由 MCU 通过 ADC 通道采集，用于赛车的运动控制策略，经过计算后 FTM 模块发出 PWM 波，驱动两舵机和直流电机对智能汽车进行转向控制和速度控制，使赛车在赛道上能够自主巡线行驶完成指定任务，并以最短的时间最快的速度跑完全程。通过编码器采集单位时间内电机转过的角度构成反馈，对电机进行了速度闭环控制，使得车辆的速度能够被精准地控制，从而完成各种变速控制，超声波模块以一定的频率发出超声波在障碍表面反射模块接收后经计算变得到了前方障碍的距离，通过检测超声波所返回的距离信息，MCU 将进行决策是否进入避障模式，控制舵机结合距离信息连续转动完成绕过障碍这一过程，此外还加入按键、显示屏和蜂鸣器进行交互方便调试。

根据以上系统方案，本系统分为七大模块，MK60DN512ZVLQ10 主控模块、电磁检测模块、距离检测模块、速度检测模块、电源稳压模块、电机驱动模块、人机交互模块。

各模块的功能作用如下所述：

MK60DN512ZVLQ10 主控模块：整车的核心控制器处理传感器采集的信息输出各类控制信号给元件达到目标需求。

电磁检测模块：完成巡线任务的核心传感器经过处理的信息能够快速准确地

显示车辆相对电磁线偏离的角度，且由于电磁板向前伸出，采集的信息具有前瞻性，为车辆的控制提供了足够的时间和次数。

距离检测模块：通过超声波的反射得到前方是否有障碍和车辆到障碍的距离，从而实现避障这一任务。

速度检测模块：得到单位时间内车轮转过的角度，实现对速度的精准控制。

电源稳压模块：从电池得到的电流经过稳压模块得到不同的电压满足车上的各类电子元件的额定电压和电流，使其稳定正常工作。

电机驱动模块：接受到 MCU 上发出的 PWM 信号生成对应大小和方向的电压施加在电机两端，实现对直流电机转矩的控制。

人机交互模块：方便人对车辆的调节，快捷改变车辆上的设定状态，跟快适应各种场地。

第三章 智能车机械结构调整与优化

3.1 电磁传感器

在室外越野电磁组的赛道上铺有一根铜导线，通有 100mA 左右 20KHz 的交变的信号电流，导线中的电流按一定规律变化时，导线周围的磁场也将发生变化，则线圈中　将感应出一定的电动势，电感与电容所谐振网络刚好选出 20KHz 的磁场强度的信号。

3.1.1 电磁传感器选择

经过分析前述理论可知，采用线圈传感器检测磁场强度的方法有很强的适应能力，并且容易实现。本设计采用的线圈传感器是工字电感，并将电感和电容串联通过谐振原理选频，输出信号较大，且能模拟出车身在赛道上距离中心导线位置的线性改变。为了提高磁场检测精度，我们对所购买的工字电感进行了严格筛选。通过公式：

$$f = \frac{1}{2\pi\sqrt{LC}} \qquad \text{式 3.1}$$

由于市面上电容规格有限，我们设计了方便匹配电感电容对的 PCB，尽量去满足最佳的 LC 数值，后来电磁杆前排更换成蓝色工字电感。

3.1.2 电磁传感器布局

由于第十四届恩智浦杯智能汽车大赛的电磁越野组是采用的 L 车摸，且无对电磁杆的限制要求，为了能保证车模尽早地感应到磁场变化以及保证磁场采集的稳定性。我们采用了双排电感的方式，前排电感为双竖直加双内八字电感排列用于转弯，提高对弯道的预测性，后排为两个水平电感（最初设计时为四个电感，后来只应用了两个）用于直道巡线，提高直道的稳定性。传感器安装方式如图 3.1。

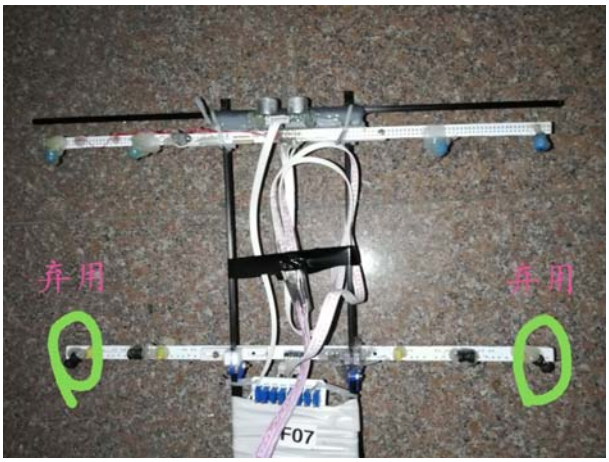


图 3.1 电磁传感器布局

3.3 速度传感器

测速装置在智能车系统中占有非常重要的地位，其要求是分辨能力强、精度高和检测时间短。从精度要求来看，增量式旋转编码器最为合适，且集成性好，抗干扰能力强。最终采用 1024 线增量式旋转编码器作为系统的测速模块。测速时，通过齿轮与电机驱动齿轮咬合，后轮一转动，FTM 脉冲计数模块就在累加编码器产生的脉冲，而后在一定时长的定时中断中将脉冲数读出，通过换算转变为后轮转速。

3.4 车模机械调整

任何的控制算法和软件程序都都有其执行的上限，越优秀的算法能够更接近这个上限然而却无法超越，因此成熟稳定的机械结构决定了设计算法的难易和效率，因此机械结构的优化和调整是必要也是重要的。

3.4.1 车模前轮转向优化

原车模前转向球头用的是活扣，如果拉杆受力形变比较大，就容易蹦出来，原装车模靠近车轮的球头放在下边，连接造成拉杆的角度太大，限制了前轮的左右转动，需要把原来球头取掉，将其装到上面，拉杆也装到上面，便可使转动角度增加 7-8°，再将连接部分下方打磨，直至前轮固定臂与转向连接臂之间无摩擦，可自由转动，至此左右转向的角度最终扩大 15°左右。前轮最小转弯半径降低至越 60cm。

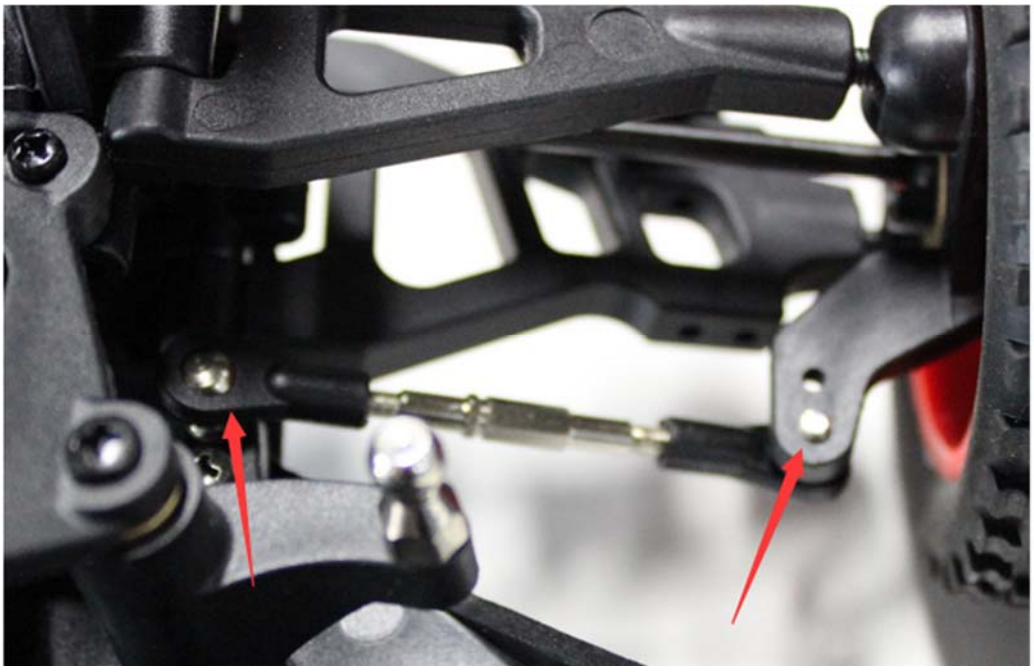


图 3.2 车模前轮转向优化

3.4.2 后轮转向改装

由于赛道均由折线连接而成，车模需要在更快更准确的转向控制才能保证在高速行车的同时稳定转入下一直道，这就对转向的性能有了高的要求。最终我们将后轮改装使后轮与前轮同时转向，前轮左转后轮右转反之亦然。如此转弯半径理论上可大幅减少。改装方案如下：

舵机臂上连接一根 20cm 的拉杆与后轮由转向柱连接，使得舵机同时控制前轮和后轮。后轮摆臂上安装抬高固定柱，便于连接联动装置，拉杆与固定板平面水平，保证不会与舵机传动杆碰撞，这样，如果车身在受力下沉，此处拉杆基本可以呈现一个水平状态，同时还保证了球头的活动空间，不会和上面的摆臂摩擦，拉杆与又转向柱连接，右转向柱通过短拉杆与转向柱连接，左右转向柱同时固定于底板之上，从而实现后轮与前轮的反向随动。

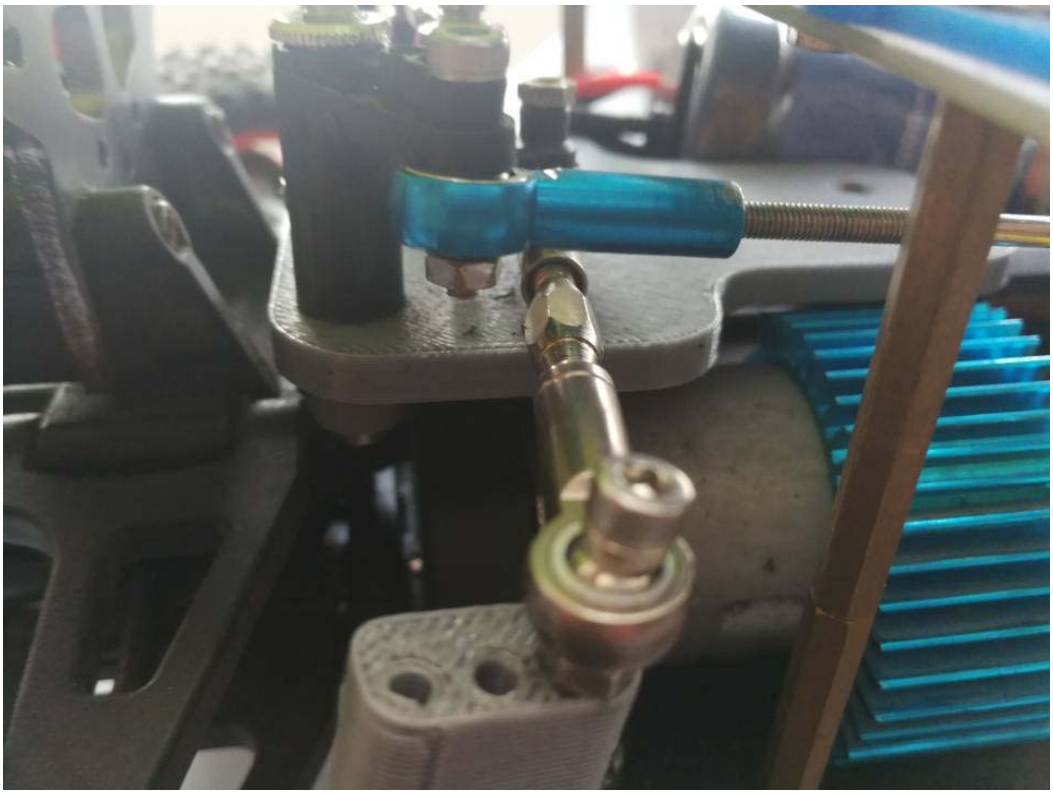


图 3.3 后轮转向改装

经测试该转后的车模转弯半径有了极为显著的变化，车模的最小转弯半径由原来的 60cm 减小到 42cm。改装后的车模极为灵活完全满足了比赛的需求但同时也增加了控制的难度，需要鲁棒性更好的控制算法。

第四章 硬件电路设计说明

根据系统的方案设计，硬件电路被分为四个模块，单片机最小系统、电源、运放和驱动。

4.1 单片机最小系统

我们采用了自制的 MK60DN512ZVLQ10 单片机最小系统为本智能车系统的核心，最小系统板引出了 144 个引脚，最小系统板上自带晶振电路、复位电路、单片机电源电路。

4.2 电源模块

板子上的所有元件都需要稳定的电源，供电的稳定直接关系到整个小车的稳定运行。 智能车供电电源又 7.2V 电池供电。板上分为电池直接供电、稳压 5V 供电、稳压 3.3V 供电、稳压可调供电，电机驱动需要电池直接供电，数字舵机需要 5-6V 电源，单片机需要 3.3V 供电，还有各种传感器等模块分别需要 3.3V 和 5V 模块电源。

5V 稳压我们采用 LM2940，LM2940 是低压差的线性稳压集成电路，芯片输入输出之间的损耗比较小，效率高，输出稳定。3.3V 稳压采用 LM2937，对于单片机来说，单片机本身功耗低，但是它对电源稳定性要求相对较高。

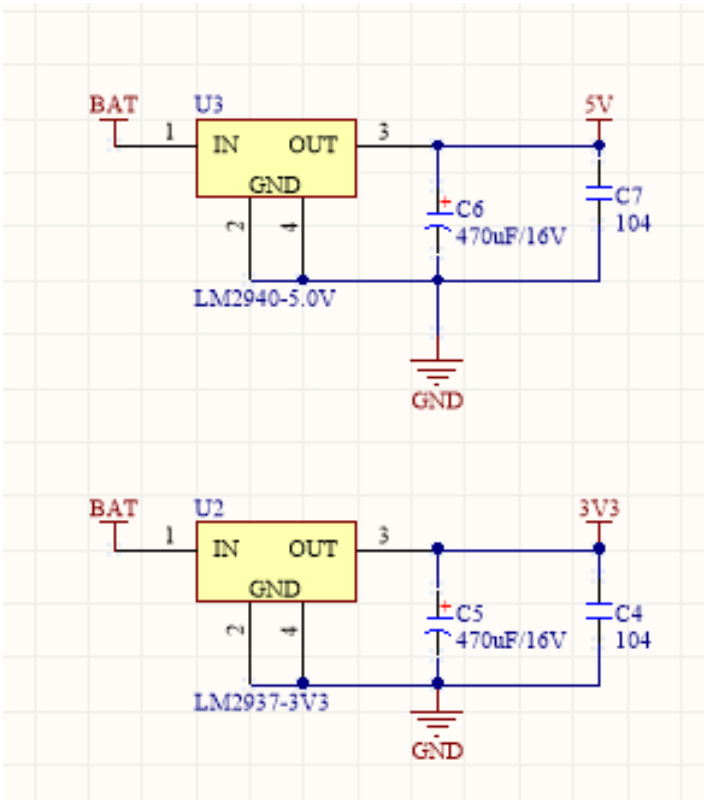


图 4.1 3.3V、5V 稳压电路

舵机工作时需要的电流较大，需要单独供电以免拉低其他稳压电路电压，。舵机可调稳压电路用 AS1015，调节电位器稳定 6V 电压供给舵机。

最终电源模块电路图如图 4.1

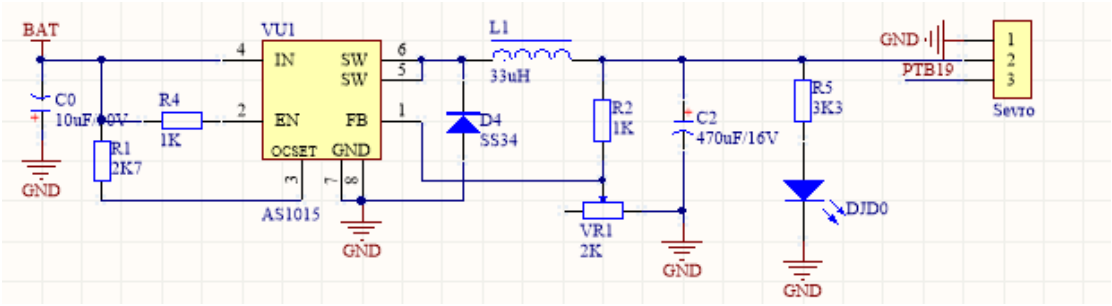


图 4.2 舵机稳压电路

4.3 运放模块

电磁传感器采集的电压幅值较小，不利于电压采样导致精度低，所以要进行放大，官方给出的如下运放用来放大电磁传感器的传来的赛道信号，并将其检波以供 AD 采集和信号处理。在制作运放的时候我们使用 OPA4377 放大电路，如图 4-3。

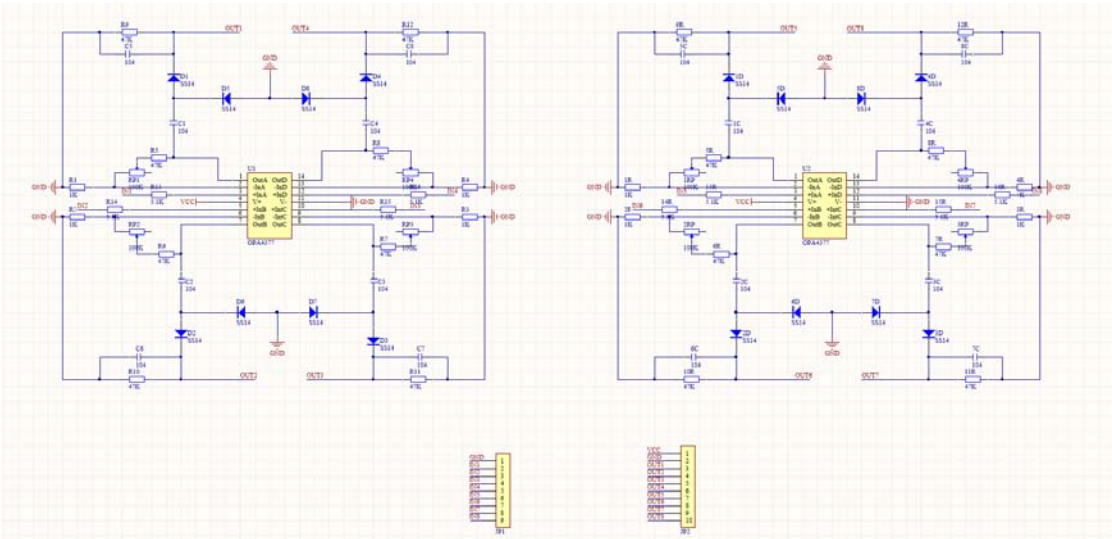


图 4.3 运放电路

4.4 电机驱动模块

驱动芯片我们采用了主流的大电流电机半桥驱动芯片 IR2104。IR2104 型半桥驱动芯片可以驱动高端和低端两个 N 沟道 MOSFET，能提供较大的栅极驱动电流，并具有硬件死区、硬件防同臂导通等功能。使用两片 IR2104 型半桥驱动

芯片可以组成完整的直流电机 H 桥式驱动电路。由于其功能完善，价格低廉容易采购，所以我们选择它进行设计，原理图如图 4.4 所示。

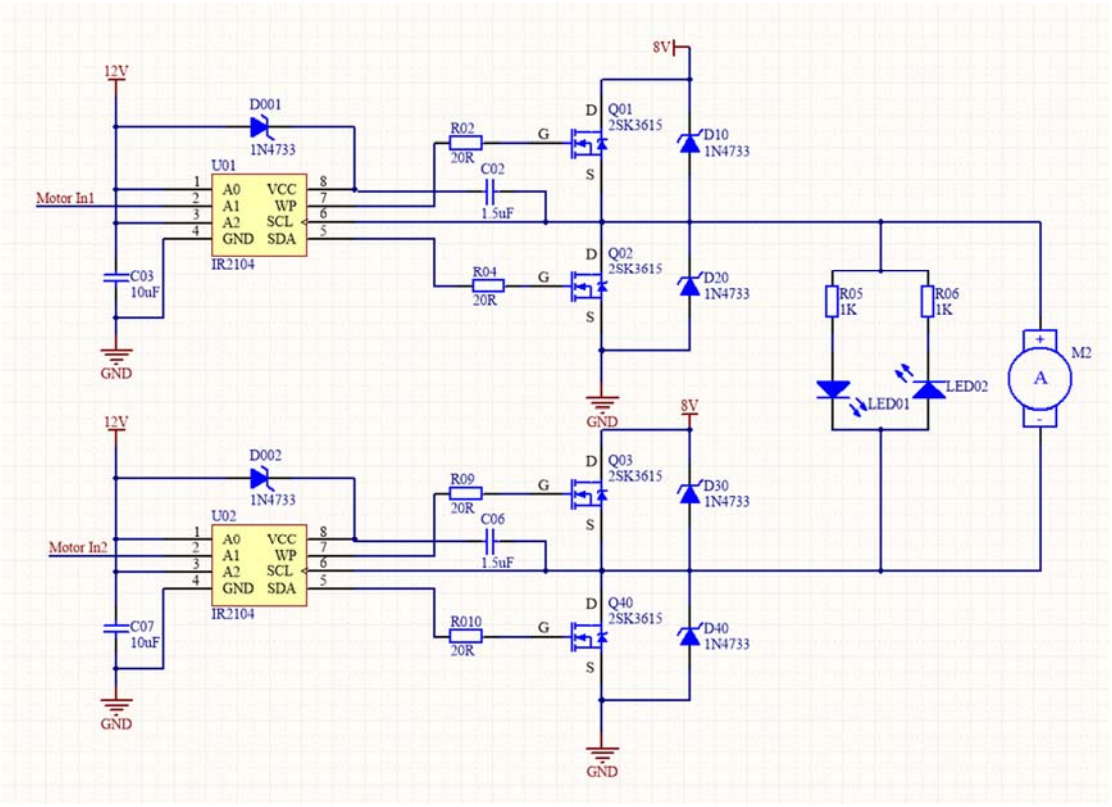


图 4.4 电机驱动模块

第五章 软件设计

5.1 软件开发平台

软件开发平台为单片机通用 IAR Embedded Workbench 开发软件。Embedded Workbench for ARM 是 IARSystems 公司为 ARM 微处理器开发的一个集成开发环境(下面简称 IAR EWARM)。比较其他的 ARM 开发环境，IAR EWARM 具有入门容易、使用方便和代码紧凑等特点 [1] 。

IAR Systems 公司目前推出的最新版本是 IAR Embedded Workbenchfor ARM version。

EWARM 中包含一个全软件的模拟程序(simulator)。用户不需要任何硬件支持就可以模拟各种 ARM 内核、外部设备甚至中断的软件运行环境。从中可以了解和评估 IAR EWARM 的功能和使用方法。

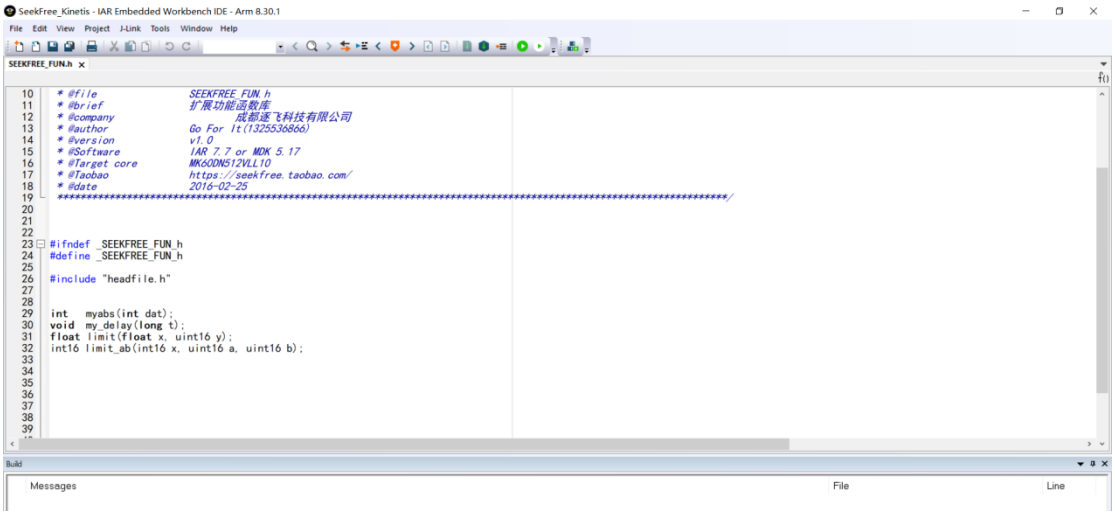


图 5.1 IAR 使用界面

5.2 软件系统总体设计

在整个系统设计中，主要用到了单片机的 5 个基本功能模块： FTM 模块、PIT 模块、AD 转换模块、GPIO 模块。通过配置寄存器先对所用到的模块进行硬件初始化，并通过相应的数据寄存器或状态寄存器的读写，实现需要的输入和输出。FTM 模块输出一定频率，占空比可调的 PWM 信号用来控制电机的运转，FTM 脉冲捕获读编码器返回的脉冲，PIT 模块用来限定程序执行的周期和顺序，使各个模块都以期望的节律工作，AD 模块将电磁运放放大的电压信号转换为数字量，GPIO 模块主要用作各种输入输出，显示、按键、串口、IIC 等等。

系统流程如图 5.2 所示。先对单片机的硬件和软件进行初始化，对传感器采集回来的数据进行整理处理，根据传感器的数据反馈给控制器，由控制算法前方路况所需的转向角度和速度。

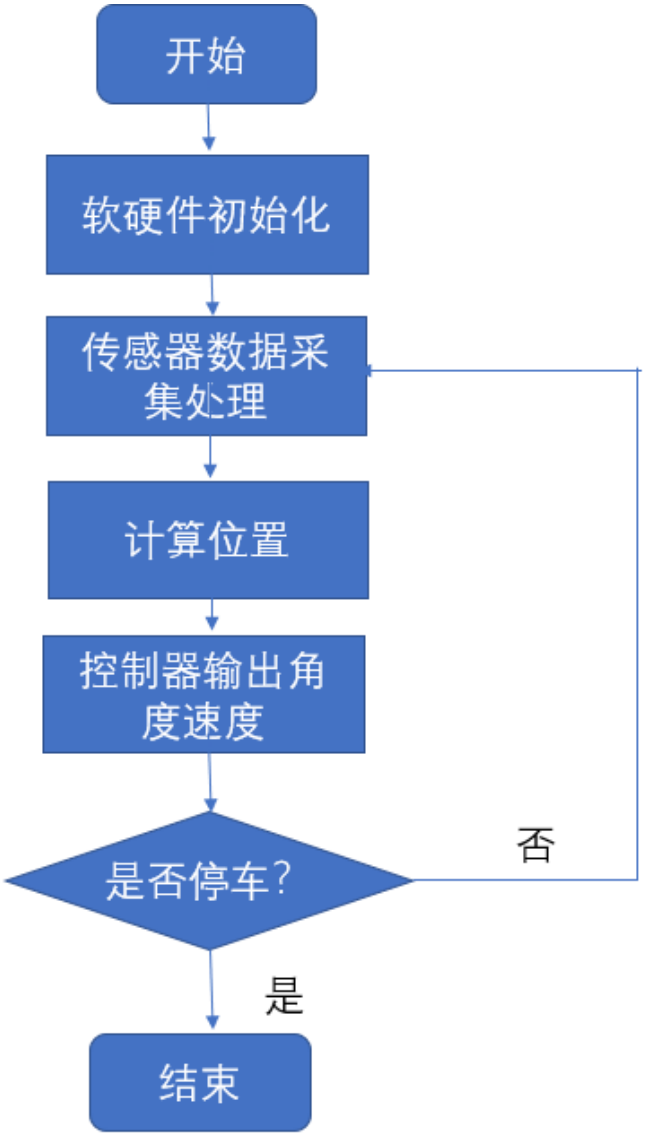


图 5.2 系统流程图

系统主程序

```
/*****************************************************************************
************************************
int main(void)
{

  //初始化
  get_clk();//上电运行获取信息设置模块参数
  ftm_motorandsevro_init();
  PIT0_init();
```

```
  ADC_init_mine();
  OLED_Init() ;
  Key_init();

  //静态显示
  OLED_P6x8Str(0,0,"<<");     OLED_P6x8Str(70,0,">>");
  OLED_P6x8Str(0,2,"<" );     OLED_P6x8Str(70,2,">" );
  OLED_P6x8Str(0,4,"=" );     //OLED_P6x8Str(72,4,"AD7");
  OLED_P6x8Str(0,6,"TD");     OLED_P6x8Str(72,6,"ST");

  while(1)
  {
  Key_scan();

  OLED_Print_Num(24,0,AD_ConvertedValues[4])                                    ;
OLED_Print_Num(88,0,AD_ConvertedValues[0]) ;
  OLED_Print_Num(24,2,AD_ConvertedValues[3])                                    ;
OLED_Print_Num(88,2,AD_ConvertedValues[1]) ;
  OLED_Print_Num(24,4,AD_ConvertedValues[2])                                    ;
//OLED_Print_Num(96,4,AD_ConvertedValues[6]) ;
  OLED_Print_Num1(24,6,(int)TurnDeviation)        ; OLED_Print_Num1(88,6,(int)State) ;
  //OLED_Print_Num1(24,6,(int)TurnDeviation     ) ; OLED_Print_Num(88,6,State) ;

  if(AD_Convertedtime_Enable==1)
  {
    Clockflag=1;
    while(My_SystemClock_10ms<AD_Convertedtime)
    {

      for(AD_5=0;AD_5<AD_Convertednum;AD_5++) //遍历五路电感采集
      { if(AD_ConvertedValues[AD_5]>AD_MaxValue[AD_5])
          AD_MaxValue[AD_5]=AD_ConvertedValues[AD_5];
        if(AD_ConvertedValues[AD_5]<AD_MinValue[AD_5])
          AD_MinValue[AD_5]=AD_ConvertedValues[AD_5];
      }
    }
    Clockflag=0;//时钟清零
    AD_Convertedtime_Enable=0;//采集使能关闭
  }
  //定时
   Clockflag=1;
```

```
  if(My_SystemClock_10ms>3000)
  {
   Speedset=0;
   Clockflag=0;//时钟清零
  }
  }
}
```

5.3 磁场信息获取

RC 谐振电路采集从赛道中心导线所产生的磁场产生的电信号，首先经过运放电路进行放大。然后对放大后的信号进行检波。我们采用传感器检测整个磁场信息和车身位置，我们采用了双排电感的方式，前排电感为双竖直加双内八字电感排列用于转弯，提高对弯道的预测性，后排为两个水平电感用于直道巡线和环岛判断，并提高直道的稳定性。

5.4 控制算法

5.4.1 转向控制——位置式 PID

在复杂的赛道上要求车体尽可能以高速稳定运行，就要求用更少的时间进行调节
PID 是比例(P)、积分(I)、微分(D)控制算法。对于转向来说不存在稳态误差因此去掉了 I 项，仅剩两个参数 Kp 和 kd。经过大量实验发现该控制器的性能差强人意，因此我们对该控制器进行了优化，将 Kp 和 Kd 根据偏差动态变化适应不同姿态下的要求。程序如下

```
 float Turn_Control(float TurnDeviation,uint8 mode)
{
  float TurnOutput,Turn_P,Turn_D;
  static float TurnDeviationLast;
  //static float TurnDeviationAbs;

 if(TurnDeviation > 0)
  {
   TurnDeviation = TurnDeviation * TurnRightKp;//调节左转与右转的对称
    //TurnDeviationAbs = TurnDeviation;//取绝对值
  }
  else
  {
    TurnDeviation = TurnDeviation * TurnLeftKp;//调节左转与右转的对称
    //TurnDeviationAbs = -TurnDeviation;//取绝对值
   }
```

```
//  //根据偏差大小进行 PD 参数分段，参数需要根据实际情况做调整
//  if(TurnDeviationAbs < TurnDeviationDivide[1])
//  {
//      TurnPID.Kp  =  (TurnDivideKp[1]-TurnDivideKp[0])*(TurnDeviationAbs-
TurnDeviationDivide[0])/(TurnDeviationDivide[1]-
TurnDeviationDivide[0])+TurnDivideKp[0];//
//      TurnPID.Kd  =  (TurnDivideKd[1]-TurnDivideKd[0])*(TurnDeviationAbs-
TurnDeviationDivide[0])/(TurnDeviationDivide[1]-
TurnDeviationDivide[0])+TurnDivideKd[0];//
//  }
//  else if(TurnDeviationAbs < TurnDeviationDivide[2])
//  {
//      TurnPID.Kp  =  (TurnDivideKp[2]-TurnDivideKp[1])*(TurnDeviationAbs-
TurnDeviationDivide[1])/(TurnDeviationDivide[2]-
TurnDeviationDivide[1])+TurnDivideKp[1];//
//      TurnPID.Kd  =  (TurnDivideKd[2]-TurnDivideKd[1])*(TurnDeviationAbs-
TurnDeviationDivide[1])/(TurnDeviationDivide[2]-
TurnDeviationDivide[1])+TurnDivideKd[1];//
//  }
//  else if(TurnDeviationAbs < TurnDeviationDivide[3])
//  {
//    TurnPID.Kp = (TurnDivideKp[3]-
TurnDivideKp[2])*(TurnDeviationAbs-
TurnDeviationDivide[2])/(TurnDeviationDivide[3]-
TurnDeviationDivide[2])+TurnDivideKp[2];//
//                          TurnPID.Kd      =      (TurnDivideKd[3]-
TurnDivideKd[2])*(TurnDeviationAbs-
TurnDeviationDivide[2])/(TurnDeviationDivide[3]-
TurnDeviationDivide[2])+TurnDivideKd[2];//
//  }
//  else
//  {
//                          TurnPID.Kp      =      (TurnDivideKp[4]-
TurnDivideKp[3])*(TurnDeviationAbs-
TurnDeviationDivide[3])/(TurnDeviationDivide[4]-
TurnDeviationDivide[3])+TurnDivideKp[3];/
//                          TurnPID.Kd      =      (TurnDivideKd[4]-
TurnDivideKd[3])*(TurnDeviationAbs-
TurnDeviationDivide[3])/(TurnDeviationDivide[4]-
TurnDeviationDivide[3])+TurnDivideKd[3];
//  }
//计算转向环 P 项
  Turn_P = TurnDivideKp[mode][0] * TurnDeviation;
```

```
  //计算速度环 D 项偏差
  Turn_D = TurnDivideKd[mode][0] * (TurnDeviation - TurnDeviationLast);
  //更新上一次偏差值
  TurnDeviationLast = TurnDeviation;
  //求和
  TurnOutput = SERVO_MID + Turn_P + Turn_D + offset;
  return TurnOutput;
}
```

5.4.2 速度控制——增量式 PID 算法

速度控制要求是对期望的设定值以最短的时间达到，超调小，稳定性好，优秀的速度控制可以做到各种可控的变速运动，是车模能够随心所欲地穿梭于赛道之上。

```
int16 Speed_Control(int16 ActualSpeed,int16 TargetSpeed)
{
  SpeedDeviation = TargetSpeed-ActualSpeed ;
  Speed_P = (int)(SPEED_KP * (SpeedDeviation-SpeedDeviationLast));
  Speed_I = (int)(SPEED_KI * SpeedDeviation);
  Speed_D = (int)(SPEED_KD * (SpeedDeviation + SpeedDeviationLastLast
- 2*SpeedDeviationLast));
  if(Speed_I > INTEGRAL_LIMIT)
  {
    Speed_I = INTEGRAL_LIMIT;
  }
  else if(Speed_I <= -INTEGRAL_LIMIT)
  {
    Speed_I = -INTEGRAL_LIMIT;
  }
  SpeedDeviationLastLast = SpeedDeviationLast;
  SpeedDeviationLast  =  SpeedDeviation;
  SpeedOutput =SpeedOutput+ Speed_P + Speed_I + Speed_D;
  if(SpeedOutput >= 999)
  {
    SpeedOutput = 999;
  }
  else if(SpeedOutput <= -999)
  {
    SpeedOutput = -999;
  }
  return SpeedOutput;
}
```

增量式 PID 要注意限幅，不然会出现溢出。

第六章 车模技术参数

主要技术参数：

项目	参数
路径检测方法（赛题组）	电磁组
车模几何尺寸（长、宽、高）（毫米）	780mm、400mm、208mm
传感器种类及个数	超声波测距模块 US-100*1、电感 10mH*8、电容 6.8nF*8、编码器 1024 线双向增量式迷你编码器*1
电机种类及个数	LS-540SM 直流电机×1
电源电压范围	7.2V—8.4V

第七章 总结

7.1 存在的问题

7.1.1 机械方面

在机械方面，前后轮的改装始终是个重点，改装的好坏直接影响了车模过弯的快速性和在直道上的稳定性，但改装后轮使后轮存在虚位，导致车模稳定性变差，加上改装后轮会使舵机负载加大，使车模更不容易控制，这都是需要不断改进的问题。

7.1.2 硬件方面

在硬件方面，电机驱动的稳定程度决定车的稳定程度，电机驱动容易因电流过大而烧坏；还有一个问题就是，电机驱动容易莫名其妙的打假赛，毫无规律可言。再有就是我们的电磁杆是 PCB 打印的，硬度不够，容易撞断，急需改进。

7.2 心得体会

智能车不像其他比赛，这个比赛几乎需要用一年的心血去完成，但是这也是这个比赛最大的迷人之处，越是来得不易，越是懂得珍惜。在这一年的奋力拼搏中，使我们更加认识到了动手能力和理论知识的重要性，而理论与实践的结合更是重中之重。由于自身理论知识的欠缺和动手能力的不佳在工作中频频受阻，走了好多弯路，但我们最终还是收获了软件硬件等各个方面的提升，收获了队友之间来自不易的友情，愿青春无限，记忆永存。

参考文献

[1] 卓晴，黄开胜，邵贝贝. 学做智能车[M]. 北京：北京航空大学出版社，2007
[2] 童诗白，华成英. 模拟电子技术基础[M]. 北京. 高等教育出版社. 2000
[3] 李仕伯，马旭，卓晴.基于磁场检测的寻线小车传感器布局研究.北京.清华大学.2009
[4] 臧杰，阎岩. 汽车构造[M]. 北京：机械工业出版社，2005
[5] 谭浩强. C 语言程序设计[M].清华大学出版社，2006.1
[6] 邵贝贝 . 单片机嵌入式应用的在线开发方法[M]. 北京：清华大学出版社， 2004
[7] 张化光，何希勤. 模糊自适应控制理论及其应用[M]. 北京：北京航空大学出版社，2002

附录 源代码

```
//-----------------------------------------
//
//            室外越野电磁组
//            2019-8-18
//            main:
//1 车
//1111111111111111111111111111111111111111111
//-----------------------------------------
#include "include.h"

int main(void)
{
   get_clk();
   ftm_motorandsevro_init();
   PIT0_init();
   ADC_init_mine();
   OLED_Init() ;
   Key_init();
   myledinit();
    myuart_init();
   //IIC_init();
   //InitMPU6050();
   //静态显示
   OLED_P6x8Str(0,0,"<^");      OLED_P6x8Str(70,0,"^>");
   OLED_P6x8Str(0,2,"<_" );    OLED_P6x8Str(70,2,"_>" );
   OLED_P6x8Str(0,4,"D0" );    OLED_P6x8Str(72,4,"LS");
   OLED_P6x8Str(0,6,"T1");      OLED_P6x8Str(72,6,"RS");
   while(1)
   {
    Key_scan();
   while((key[2]==0)&&(AD_Convertedtime_Enable==1))
   {
         Key_scan();
         for(AD_Num=0;AD_Num<AD_Convertednum;AD_Num++) //遍历五路电感采集
         {
            if(AD_ConvertedValues[AD_Num]>AD_MaxValue[AD_Num])
               AD_MaxValue[AD_Num]=AD_ConvertedValues[AD_Num];
            if(AD_ConvertedValues[AD_Num]<AD_MinValue[AD_Num])
               AD_MinValue[AD_Num]=AD_ConvertedValues[AD_Num];
         }
         OLED_Print_Num1(24,0,AD_ConvertedValues[LF])                                    ;
```

```
OLED_Print_Num1(88,0,AD_ConvertedValues[RF]) ;
        OLED_Print_Num1(24,1,AD_ConvertedValues[LS])                                    ;
OLED_Print_Num1(88,1,AD_ConvertedValues[RS]) ;
        OLED_Print_Num1(24,2,AD_ConvertedValues[LA])                                    ;
OLED_Print_Num1(88,2,AD_ConvertedValues[RA]) ;
        OLED_Print_Num1(24,3,AD_ConvertedValues[LA2]);
OLED_Print_Num1(88,3,AD_ConvertedValues[RA2]) ;
        OLED_Print_Num1(88,6,(int)State) ;
   }
   if(key[2]==1)
   {
                AD_Convertedtime_Enable=0;
         //OLED_P6x8Str(0,0,"RUN");

        if(CSB_Count>30)
        {
          Distance_cm=uart_distance_caluate();
          CSB_Count=0;
         }

//                                          OLED_Print_Num1(24,1,(int)Distance_cm)      ;
OLED_Print_Num1(88,1,(int)State);
   }
   else if(key[2]==2)
   {

         OLED_Print_Num1(24,0,(int)AD_Normalized[LF])                                   ;
OLED_Print_Num1(88,0,(int)AD_Normalized[RF]) ;
         OLED_Print_Num1(24,1,(int)AD_Normalized[LS])                                   ;
OLED_Print_Num1(88,1,(int)AD_Normalized[RS]) ;
         OLED_Print_Num1(24,2,(int)AD_Normalized[LA])                                   ;
OLED_Print_Num1(88,2,(int)AD_Normalized[RA]) ;
                                                                                        ;
OLED_Print_Num1(88,3,(int)State)                ;
         OLED_Print_Num1(24,5,(int)TurnDeviation[0])                                    ;
OLED_Print_Num1(88,5,(int)TurnDeviation[1]);
         OLED_Print_Num1(24,6,(int)ServoPWM);
OLED_Print_Num1(88,6,(int)(FuzzyKf*100));
         OLED_Print_Num1(24,7,(int)Distance[2]/400);

   }
   }
}
```

```
//            isc:
void PIT1_IRQHandler(void)
{
   static uint8 PIT1_Count = 0;
   //static uint16 LL_Count =0;
   PIT_FlAG_CLR(pit1);
   PIT1_Count++;
   //My_SystemClock_10ms=My_SystemClock(Clockflag,PIT1_Count); //中断驱使时钟
   AD_ConvertedValues[0]=   adc_once(ADC0_SE8,ADC_12bit);
   AD_ConvertedValues[1]=   adc_once(ADC0_SE9,ADC_12bit);
   AD_ConvertedValues[2]=   adc_once(ADC0_SE12,ADC_12bit);
   AD_ConvertedValues[3]=   adc_once(ADC0_SE13,ADC_12bit);
   AD_ConvertedValues[4]=   adc_once(ADC1_SE14,ADC_12bit);
   AD_ConvertedValues[5]=   adc_once(ADC1_SE15,ADC_12bit);
   AD_ConvertedValues[6]=   adc_once(ADC1_SE16,ADC_12bit);
   AD_ConvertedValues[7]=   adc_once(ADC0_SE16,ADC_12bit);
   Anti_Interference_Filter(AD_ConvertedValues,AD_AfterFiter1,5);
   ADC_Normalization(AD_AfterFiter1,AD_Normalized);//数据归一化
   FuzzyKf=PL_Calculate(AD_Normalized);
   //FuzzyLf=Cross_Calculate(AD_Normalized);
   //judege90_K=judege90(AD_Normalized,1.12,0.5);
   TurnDeviation[0] = Deviation_Calculate(AD_Normalized,0);
   TurnDeviation[1] = Deviation_Calculate(AD_Normalized,1);
   TurnDeviation[2] = Deviation_Calculate(AD_Normalized,2);


//   RG_PD=(int)HD();
//

   if(AD_Convertedtime_Enable==0)
   State=Analysis(AD_Normalized,AD_AfterFiter1,Distance_cm,Distance,TurnDeviation);//分析
   if(State==CL) {myledbright(); }
  //                 else                  if((AD_AfterFiter1[LA2]<348)||(AD_AfterFiter1[LA2]>3748))
{gpio_set(E1,1);gpio_set(E1,1);led0();}
   else if((State==OA))     {gpio_set(E1,1);}
   else if((State==RG))     {gpio_set(E1,1);led0();}
   else if((State==LA69)||(State==RA69))     {gpio_set(E1,1); myledbright();}
   else if(State==ED)   {myledbright();}
   else     {gpio_set(E1,0);led1();}
//   if(Distance[2]>480000)//调试用
   //State=OA;
   if(PIT1_Count>10)//转向闭环周期为 20ms
   {
      PIT1_Count = 0;
```

```
//舵机控制


        if(State==FBPL) //双排混合
        {
                ServoPWM = (int) Doublepass_Turn_Control(TurnDeviation,FuzzyKf);//转向闭环
控制
                ftm_sevro_control(158);
                speedoutput=(int)speedchange();
        }
        else if(State==FPL) //前排
        {
                ServoPWM = (int)Turn_Control(TurnDeviation,0);//转向闭环控制
                ftm_sevro_control(ServoPWM);
                speedoutput=(int)speedchange();

        }
        else if(State==BPL) //后排
        {
                ServoPWM = (int)Turn_Control(TurnDeviation,1);//转向闭环控制
                ftm_sevro_control(ServoPWM);
                speedoutput=(int)speedchange();

        }




        else if(State==RG)//环岛
        {
                ServoPWM = (int)  Doublepass_Turn_Control(TurnDeviation,FuzzyKf);
                //左左
                if(RGdirection==0)          {if(RGtimes==0){ServoPWM =ServoPWM+11;}else
if(RGtimes==1)ServoPWM =ServoPWM-9;else if(RGtimes==2)ServoPWM =ServoPWM+11;else
if(RGtimes==3)ServoPWM =ServoPWM-9;}
                //左右
                else if(RGdirection==1){if(RGtimes==0){ServoPWM =ServoPWM+11;}else
if(RGtimes==1)ServoPWM =ServoPWM-9;else if(RGtimes==2){ServoPWM =ServoPWM-
11;}else if(RGtimes==3)ServoPWM =ServoPWM+9;}
                //右左
                else if(RGdirection==2){if(RGtimes==0){ServoPWM =ServoPWM-11;}else
if(RGtimes==1)ServoPWM =ServoPWM+9;else if(RGtimes==2)ServoPWM
=ServoPWM+11;else if(RGtimes==3)ServoPWM =ServoPWM-9;}
```

```
                //右右
                else    if(RGdirection==3){if(RGtimes==0){ServoPWM    =ServoPWM-11;}else
if(RGtimes==1)ServoPWM =ServoPWM+9;else if(RGtimes==2)ServoPWM =ServoPWM-11;else
if(RGtimes==3)ServoPWM =ServoPWM+9;}
                ftm_sevro_control(ServoPWM);
                speedoutput=Speedset;
        }
        else if(State==LA69) //左直角
        {         myledbright();
                ServoPWM = SERVO_MAX_RIGHT;
                ftm_sevro_control(ServoPWM);
                speedoutput=Speedset;
        }
        else if(State==RA69)//右直角
        {         myledbright();
                ServoPWM = SERVO_MIN_LEFT;
                ftm_sevro_control(ServoPWM);
                speedoutput=Speedset;
        }
                else if(State==SF)//急停车
        {
                ftm_sevro_control(ServoPWM);
                speedoutput=0;
        }
                else if(State==OA)//避障
        {

           ServoPWM=(int)Obstacle_avoidance(Distance[1]);
           ftm_sevro_control(ServoPWM);
           if(Distance[1]<OA_Distance1)
              speedoutput=-20;
              else speedoutput=(int)speedchange();
        //else speedoutput=100;
        }
        else if(State==CL)//防撞
        {
              ServoPWM = (int) Doublepass_Turn_Control(TurnDeviation,FuzzyKf);//转向闭环
控制
                ftm_sevro_control(ServoPWM);
                speedoutput=Speed_Control_Avoid(Distance_cm);
        }
     }
   //结束舵机控制
//电机控制
```

```
    Nowspeed=ftm_quad_get(ftm1);
    if(AD_Convertedtime_Enable==0)
    {
    if(State!=RG)CSB_Count++;
    ftm_motor_control(Speed_Control(Nowspeed,speedoutput));
    }
    if(Nowspeed>0)//总长
       Distance[2]=Distance[2]+Nowspeed;

     if(State==RG)//环岛
    {
       if(Nowspeed>0)
       Distance[0]=Distance[0]+Nowspeed;
    }
       if(State==OA)//避障
    {
       if(Nowspeed>0)
       Distance[1]=Distance[1]+Nowspeed;
       if(Nowspeed<0)
       Distance[1]=Distance[1]-Nowspeed;
    }
     if(State==CL)//防撞
       {
     if(Nowspeed<0)
       Distance[4]=Distance[4]-Nowspeed;
    }
       if((State==LA69)||(State==RA69))//直角
    {
       if(Nowspeed>0)
       Distance[3]=Distance[3]+Nowspeed;
    }

if((State!=OA)&&(State!=RG)&&(State!=LA69)&&(State!=RA69)&&(State!=CL)&&(State!=R
GOUT))
    {
       Distance[0]=0;
       Distance[1]=0;
       Distance[3]=0;
       Distance[4]=0;
       }

    ftm_quad_clean(ftm1);

}
```